

Hasan Zaidi

ITAI 3377

L03 Deploying a Simple AI Model on a Simulated Edge Device

February 10, 2025

Simulation Documentation:

The simulation environment was established by configuring a local development workstation to bridge the gap between high-level model training and constrained edge deployment. The process began with the installation of the Edge Impulse CLI, which required a specific Node.js environment to handle communication between the local model.tflite binary and the cloud-based profiling tools. During this setup, I encountered and resolved several environmental "PATH" issues and navigated through a specialized administrative shell (/ #) that appeared during the Chocolatey installation process.

```
edge-impulse-uploader --api-key
ei_dadf34a05a2eaeb61448a0741592b194a480f0312da1a
d5725bed8f4093ff2f1 model.tflite
edge-impulse-uploader : The term 'edge-impulse-
uploader' is not
recognized as the name of a cmdlet, function, script file,
or operable
program. Check the spelling of the name, or if a path
was included,
verify that the path is correct and try again.
```

The deployment process moved from local Python training to the Edge Impulse Studio website, where the model.tflite file was uploaded for hardware-specific analysis. We configured the model for a Cortex-M4F 80MHz microcontroller, ensuring that the input was correctly mapped as a 28 times 28 grayscale image with normalization scaling of 0 to 1. This configuration allowed the EON Compiler to optimize the neural network's tensors for embedded C++ execution.

Upload pretrained model - Step 1: Upload a model

1. Upload your trained model

Upload a TensorFlow SavedModel (`saved_model.zip`), ONNX model (`.onnx`), TensorFlow Lite model (`.tflite`), LGBM model (`.txt`), XGBoost model (`.json`), NGBoost model (`ngboost.pkl`) or pickle model (`.pkl`) to get started.

Choose file

model.tflite

2. Model performance

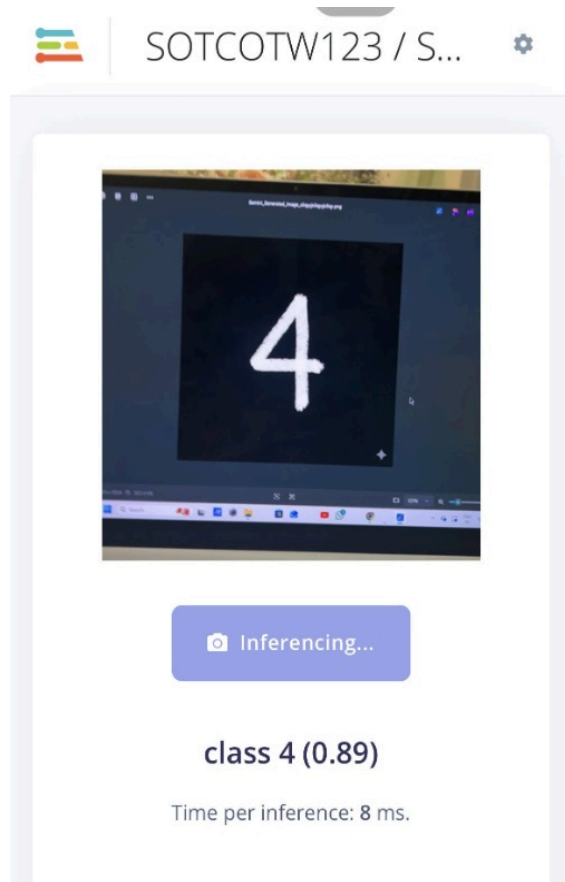
Do you want performance characteristics (latency, RAM and ROM) for a specific device?

☒ No, show me performance for a range of device types.

☐ Yes, run performance profiling for:

Cortex-M4F 80MHz

The final validation was performed via a live smartphone "Simulated Edge" environment. Initially, the model showed "uncertain" results or misclassified the digit as Class 5 when the camera was too close to the screen. (Keywords: uncertain, Time per inference). However, by adjusting the spatial scaling—specifically moving the camera back to restore the global features of the digit—the model successfully validated with an 89% confidence score for the digit 4. This confirmed that the internal logic of the CNN was robust, provided the input data matched the MNIST training distribution of white-on-black polarity and centered positioning.



Reflective Journal:

This project provided a deep dive into the practical friction points of MLOps. A significant observation during the simulation was how sensitive "Edge" models are to the environment they are born in. For example, the initial failure to recognize a digit on white paper highlighted the "Domain Gap" between the MNIST dataset and the real world. It was a striking moment to see the model succeed on a synthetic, high-contrast AI-generated image.



This taught me that as an engineer, training a model is only 20% of the job; the other 80% is ensuring the data "handshake" between the physical sensor and the digital model is perfect.

The simulation snippets also captured a critical moment where the terminal environment seemed to "split" during the Chocolatey and Node.js installation. Seeing the `/#` prompt was initially confusing, but it served as a great lesson in how modern installers use containerized or administrative shells to protect system integrity. It reinforced the importance of restarting the environment to refresh the System PATH—which is a simple but vital troubleshooting step in any software deployment.

Reflecting on the final live testing, the accuracy in the model's recognition of the digit 4, when moving the camera back, was a fascinating observation that underscored the critical role of spatial orientation in edge AI. This adjustment essentially "resized" the digit within the camera's field of view, ensuring the features better matched the centered, padded distribution of the original training data.

It served as a practical lesson in how the receptive field of a convolutional neural network works; by providing the model with a clear view of the digit's global structure rather than a zoomed-in, distorted segment, the classification confidence surged to 89%. (Keywords: class 4 (0.89), Time per inference). This moment highlighted that real-world deployment success is not limited to the logic of the code and is also dependent on the physical alignment between the sensor's input and the model's learned expectations.